

MỤC LỤC

MỤC LỤC.....	1
Tuần 4: Message Queue.....	2
1. Tổng quan về Message Queue.....	2
1.1. Định nghĩa.....	2
1.2. Cách hoạt động của Message Queue.....	2
1.3. Các thành phần chính trong một hệ thống MQ.....	2
2. Tại sao cần dùng Message Queue?.....	3
2.1. Tách rời hệ thống.....	3
2.2. Tăng khả năng chịu tải.....	3
2.3. Xử lý bất đồng bộ.....	3
2.4. Tăng độ bền và độ tin cậy.....	3
2.5. Nâng cao khả năng mở rộng.....	3
2.6. Retry, xử lý lỗi và bảo đảm không mất dữ liệu.....	4
3. Các loại Message Queue phổ biến.....	4
3.1. RabbitMQ.....	4
3.2. Apache Kafka.....	4
3.3. AWS SQS.....	4
3.4. Google Pub/Sub.....	5
3.5. Redis Streams / Redis PubSub.....	5

Tuần 4: Message Queue

(Trong nhiệm vụ tuần 4 này, em xin phép chưa hoàn thiện ở phần Implement a, em dừng lại ở việc tìm hiểu Message Queue a)

1. Tổng quan về Message Queue

1.1. Định nghĩa

Message Queue (MQ) là một cơ chế truyền thông tin giữa các hệ thống thông qua hàng đợi tin nhắn. Trong mô hình này, các ứng dụng gửi dữ liệu dưới dạng message vào một queue, và các ứng dụng khác sẽ đọc message khi chúng sẵn sàng xử lý. MQ giúp tách biệt quá trình gửi và nhận dữ liệu, cho phép các hệ thống giao tiếp với nhau mà không phải hoạt động đồng thời. Đây là nền tảng quan trọng trong các hệ thống phân tán và kiến trúc microservices.

1.2. Cách hoạt động của Message Queue

Message Queue hoạt động dựa trên mô hình Producer - Queue - Consumer. Producer gửi message vào hàng đợi. Queue lưu trữ message theo thứ tự, đảm bảo message không bị mất kể cả khi hệ thống gặp sự cố. Consumer sẽ nhận và xử lý message theo tốc độ của riêng nó. Các MQ hiện đại hỗ trợ nhiều chế độ tiêu thụ, như xử lý theo nhóm, xử lý tuần tự hoặc phân phối theo nhiều consumer để tăng tốc. Cơ chế này giúp hệ thống không bị nghẽn khi có lượng lớn dữ liệu cần xử lý.

1.3. Các thành phần chính trong một hệ thống MQ

Một hệ thống Message Queue thường bao gồm bốn thành phần chính. Producer là nơi tạo và gửi message. Queue là nơi lưu trữ tạm thời message, đảm bảo có thể truyền dữ liệu theo đúng thứ tự hoặc đúng cấu hình. Consumer là thành phần xử lý message, có thể có một hoặc nhiều consumer tùy tải hệ thống. Broker là dịch vụ trung gian điều phối toàn bộ quá trình gửi, lưu trữ, phân phối và đảm bảo message được xử

lý an toàn. Sự phối hợp của các thành phần này tạo nên một hệ thống truyền tin linh hoạt, tin cậy và dễ mở rộng.

2. Tại sao cần dùng Message Queue?

2.1. Tách rời hệ thống

MQ cho phép các hệ thống giao tiếp mà không phụ thuộc thời gian thực vào nhau. Producer không cần biết Consumer đang chạy hay không, chỉ cần gửi message vào queue. Điều này giúp các microservice tách biệt, dễ triển khai và dễ bảo trì.

2.2. Tăng khả năng chịu tải

Trong trường hợp lượng yêu cầu tăng đột biến, MQ đóng vai trò như một bộ đệm. Producer vẫn có thể tiếp tục gửi message, còn Consumer sẽ xử lý từng message theo tốc độ ổn định. Nhờ đó hệ thống tránh bị quá tải và duy trì hiệu năng ổn định.

2.3. Xử lý bất đồng bộ

Nhiều tác vụ không cần thực hiện ngay lập tức. MQ cho phép chuyển những việc này sang xử lý bất đồng bộ bằng cách đẩy message vào queue. Điều này giúp hệ thống phản hồi người dùng nhanh hơn, đặc biệt với các tác vụ tốn thời gian như gửi email, xử lý ảnh hoặc tính toán phức tạp.

2.4. Tăng độ bền và độ tin cậy

Message Queue có khả năng lưu trữ message một cách bền vững. Khi Consumer bị lỗi hoặc hệ thống gặp sự cố, message vẫn nằm trong queue và không bị mất. MQ còn hỗ trợ cơ chế xác nhận đã xử lý thành công, giúp đảm bảo toàn vẹn dữ liệu và tránh việc xử lý trùng lặp.

2.5. Nâng cao khả năng mở rộng

MQ cho phép dễ dàng mở rộng hệ thống bằng cách tăng số lượng Consumer để xử lý song song nhiều message hơn. Bản thân hạ tầng MQ cũng có thể chạy phân tán để đáp ứng lưu lượng lớn. Điều này rất phù hợp cho các ứng dụng có nhu cầu tăng trưởng nhanh hoặc tải biến động mạnh.

2.6. Retry, xử lý lỗi và bảo đảm không mất dữ liệu

MQ hỗ trợ tự động retry khi message xử lý thất bại, đồng thời cung cấp dead-letter queue để lưu trữ các message lỗi không thể xử lý. Nhờ đó, hệ thống vừa đảm bảo an toàn dữ liệu vừa giúp dễ dàng theo dõi và phân tích lỗi. Đây là lý do MQ được sử dụng rộng rãi trong các hệ thống yêu cầu độ tin cậy cao.

3. Các loại Message Queue phổ biến

3.1. RabbitMQ

RabbitMQ là một MQ mạnh mẽ dựa trên giao thức AMQP. Nó hỗ trợ nhiều mô hình routing như fanout, topic, direct, giúp linh hoạt trong truyền message. RabbitMQ phù hợp với các hệ thống yêu cầu độ tin cậy và khả năng xử lý message phức tạp.

3.2. Apache Kafka

Kafka được thiết kế cho xử lý dữ liệu streaming với tốc độ rất cao. Khác với MQ truyền thống, Kafka lưu trữ message dạng log phân tán, cho phép nhiều consumer đọc cùng một dữ liệu. Kafka phù hợp cho phân tích dữ liệu thời gian thực, tracking sự kiện hoặc xây dựng pipeline dữ liệu lớn.

3.3. AWS SQS

AWS SQS là dịch vụ MQ dạng cloud, không cần quản lý server. SQS đảm bảo độ bền và khả năng mở rộng tự động. Nó hỗ trợ standard queue cho throughput cao và FIFO queue cho xử lý tuần tự. SQS đặc biệt phù hợp cho môi trường serverless và hệ thống chạy trên AWS.

3.4. Google Pub/Sub

Google Pub/Sub là dịch vụ nhắn tin phân tán giúp truyền dữ liệu ở quy mô lớn. Pub/Sub hỗ trợ mô hình publish-subscribe mạnh mẽ, phù hợp cho các hệ thống event-driven và pipeline dữ liệu. Nó tích hợp tốt với các dịch vụ của Google Cloud.

3.5. Redis Streams / Redis PubSub

Redis Streams là MQ nhẹ, hiệu năng cao, thích hợp cho các use case thời gian thực. Redis PubSub hỗ trợ truyền thông điệp theo mô hình publish-subscribe nhưng không lưu trữ message lâu dài. Cả hai đều phù hợp cho hệ thống cần tốc độ cao và độ trễ thấp.